PORTFOLIO PROJECT · COMPLETE GUIDE

Enterprise Network Infrastructure with Cisco & Linux

Design, build, secure, automate, monitor and verify a two-site enterprise network. This guide covers every step in order, every tool and what it is for, the exact commands, the results, the problems solved, and a full interview-preparation section.

22 / 22	10 / 10	23	30+
automated tests pass	troubleshooting challenges	real problems solved	tools & technologies

Repository: github.com/muttakin210/Enterprise-Network-Infrastructure-Cisco-Linux
Author: muttakin210 · **Built:** 9 November 2026 · **Host:** Windows 11 PC (16 GB), VirtualBox 7.2, Cisco Packet Tracer 9.0

Contents

1. Project at a glance
2. Network design (topology, addressing, policy)
3. Build order: what was done, step by step
4. Tool reference: what each tool is and does
5. Command cheat sheet
6. Rebuild the lab from zero
7. Test results and troubleshooting challenges
8. Problems hit and how they were solved
9. Security decisions
10. Limitations and honest scope
11. Interview preparation
12. Glossary

1. Project at a glance

Scenario. A company has a **Head Office (HQ)** and one **Branch**. HQ hosts a shared Linux server. Departments are separated by VLANs. The two sites are joined by a point-to-point WAN link running OSPF. The job is to design, build, secure, automate, monitor and *prove* the whole network with test evidence.

Area	What was delivered
Cisco routing & switching	Sanitized IOS configs for 2 routers and 2 switches: 7 VLANs, 802.1Q trunks, router-on-a-stick, OSPF area 0, DHCP relay, SSH-only management, ACLs, port security, DHCP snooping, BPDU Guard, Rapid-PVST, SNMPv3 and Syslog. Built and verified in **Cisco Packet Tracer 9.0**.
Linux services (srv01)	Ubuntu Server 22.04 VM: **BIND9** DNS, **Kea** DHCP, **Nginx** intranet, **rsyslog** central logging, **Net-SNMP** v3 agent, **UFW** firewall, SSH hardening, all built by one idempotent script.
End-to-end emulation	The whole topology rebuilt on the VM with Linux network namespaces, VLAN-aware bridges, **FRR** (OSPF), **dhcrelay** and **nftables** ACLs, so that real routed tests can run without licensed Cisco images.
Automation	**Ansible** backs up and sanitizes device configs and checks OSPF. A **Python** health check (ping and TCP/22 → CSV log) comes with unit tests.
Monitoring	**LibreNMS** and **Zabbix** poll every device over SNMPv3 authPriv. One **Grafana** dashboard combines both. Syslog captures link and OSPF events.
Remote access	**WireGuard** VPN on the HQ edge. Admins reach only servers and management networks, and a router-side filter blocks Finance.
Documentation & audit	**NetBox** is the source of truth, loaded by API. An **Nmap** audit compares the live network with NetBox and checks the firewall and ACL policy.
Verification	22 automated tests with saved raw evidence, 10 troubleshooting challenges, and **tcpdump/tshark/Wireshark** captures of OSPF, DHCP relay, SNMPv3, WireGuard and Syslog.

Final results

> **Tests:** T01–T22 all **PASS**. Reachability, DHCP relay, OSPF, inter-VLAN and WAN routing, web, DNS, both ACLs, WAN failure, Syslog, SNMPv3, Ansible, Python, hardening, services, WireGuard, LibreNMS, Zabbix, NetBox, Grafana, Nmap and packet analysis.

> **Troubleshooting:** 10/10 injected faults found and fixed: wrong VLAN, shut subinterface, missing OSPF network, hello/dead mismatch, pruned VLAN, firewall block, missing helper-address, wrong ACL order, SSH blocked, SNMP key mismatch.

> **OSPF re-convergence** after a WAN failure was measured at **11.0 s**. The Syslog server recorded `Full → Deleted` and then `Loading → Full` for the neighbor 2.2.2.2.

Repository layout

```
cisco/        routers/ switches/ (IOS configs) · ospf/ · security/ · packet-tracer/ (.pkt + generated
configs)
linux/        setup-server.sh · netplan/ dns/ dhcp/ nginx/ firewall/ syslog/ snmp/ · wireguard/
automation/   ansible/ (inventory, vault example, backup + verify-ospf playbooks) · python/
(health_check.py + tests)
monitoring/   snmp/ syslog/ · librenms/ · zabbix/ · grafana/ (provisioned)
ipam/netbox/  install + API import of the addressing plan
testing/      emulation/ (netns-lab.sh, wireguard-lab.sh) · run-tests.sh · results/ · troubleshooting/
              nmap/ · capture/ · screenshots/
topology/     addressing-plan.csv/.xlsx · enterprise-topology.png
docs/         build-log.md · limitations.md · cv-entry.md · this guide
```

2. Network design

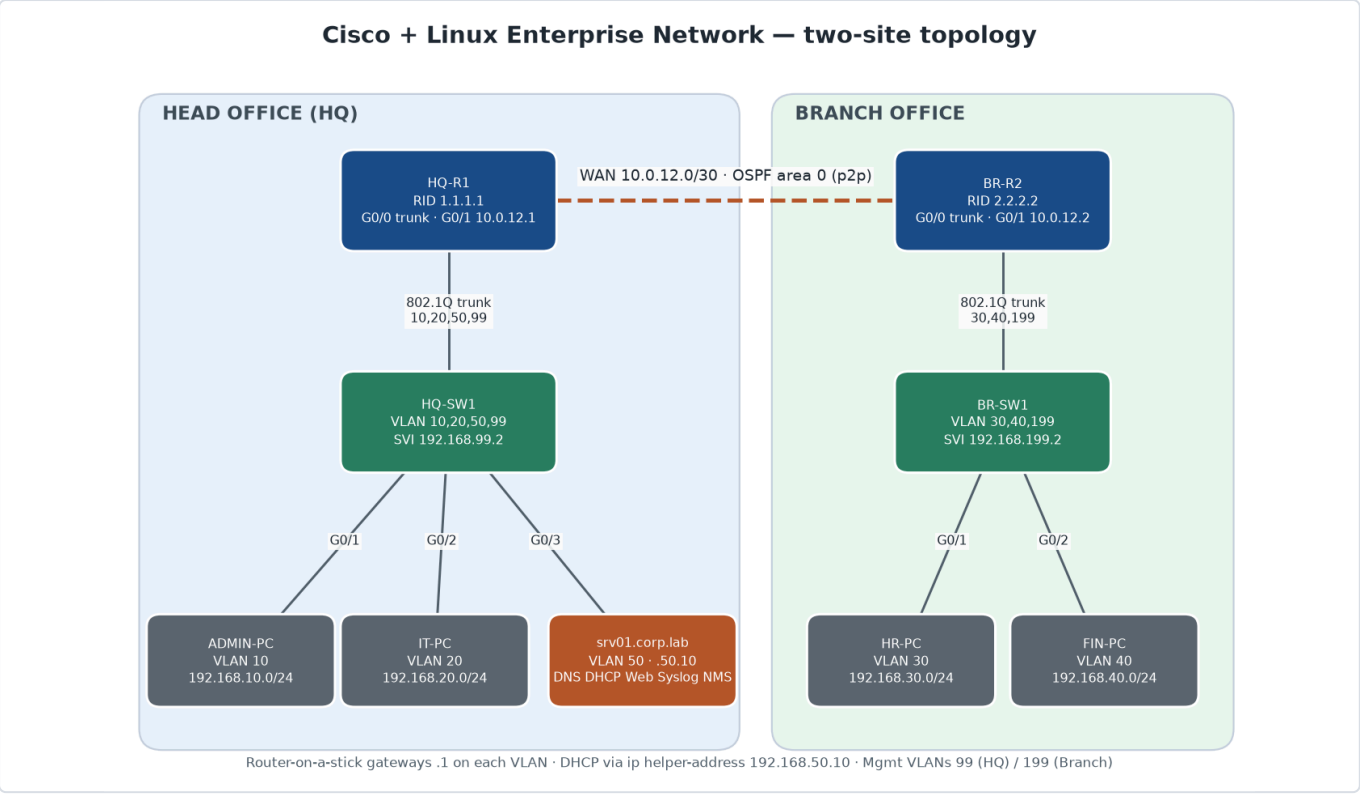


Figure 1. Two-site topology: HQ-R1 and BR-R2 joined by a /30 WAN (OSPF area 0), each with an access switch carrying department VLANs.

Addressing plan

Site	VLAN	Name	Subnet	Gateway	Notes
HQ	10	ADMIN	192.168.10.0/24	192.168.10.1 (HQ-R1 G0/0.10)	DHCP via relay; full access
HQ	20	IT	192.168.20.0/24	192.168.20.1 (HQ-R1 G0/0.20)	DHCP via relay; blocked from Finance and management
HQ	50	SERVERS	192.168.50.0/24	192.168.50.1 (HQ-R1 G0/0.50)	srv01 = 192.168.50.10 (static)
HQ	99	MANAGEMENT	192.168.99.0/24	192.168.99.1 (HQ-R1 G0/0.99)	HQ-SW1 SVI = 192.168.99.2
Branch	30	HR	192.168.30.0/24	192.168.30.1 (BR-R2 G0/0.30)	DHCP via relay; blocked from Finance
Branch	40	FINANCE	192.168.40.0/24	192.168.40.1 (BR-R2 G0/0.40)	Protected by FINANCE-PROTECT
Branch	199	MANAGEMENT	192.168.199.0/24	192.168.199.1 (BR-R2 G0/0.199)	BR-SW1 SVI = 192.168.199.2
WAN	–	R1–R2	10.0.12.0/30	HQ-R1 .1 / BR-R2 .2	OSPF point-to-point
Both	999	PARKING	–	–	Native VLAN of trunks and unused ports (shut)
VPN	–	WireGuard	10.99.0.0/24	HQ-R1 wg0 10.99.0.1	Remote admins, advertised in OSPF

DHCP pools are .100–.199 in each client VLAN. Router IDs: HQ-R1 1.1.1.1, BR-R2 2.2.2.2. The internet link used by the VPN test is 203.0.113.0/30 (TEST-NET-3).

Access policy

From \ To	Admin 10	IT 20	Servers 50	Mgmt 99/199	HR 30	Finance 40
Admin 10	–	✓	✓	✓	✓	✓
IT 20	✓	–	✓	✗ MGMT-PROTECT	✓	✗ FINANCE-PROTECT
HR 30	✓	✓	✓	✗ MGMT-PROTECT	–	✗ FINANCE-PROTECT
srv01	✓	✓	–	✓	✓	✓
VPN admins	✗	✗	✓	✓	✗	✗ VPN-FILTER

Design decisions.

- **Router-on-a-stick:** one trunk carries all VLANs to the router, which routes between subinterfaces. It is simple, cheap and fine for a small site.
- **Point-to-point OSPF** on the /30 WAN: no DR/BDR election and faster adjacency. **passive-interface default** keeps hellos off the LANs.
- **Central DHCP** on srv01 with `ip helper-address` : one Kea server for every VLAN. Kea picks the pool from the relay address (giaddr).
- **Native VLAN 999** carries no hosts. This removes VLAN hopping through native-VLAN double tagging.
- **Outbound ACLs on the protected subinterface**, close to the destination, so one ACL protects the whole VLAN.

3. Build order: what was done, step by step

Steps are listed in the order they were done. Each step says what was done, which tools were used, why, and the key commands. Work was committed to GitHub in stages.

Commit	Time	Stage
f905f7e	03:13	Cisco configs, security controls, OSPF design, addressing plan
89c0be1	03:17	Ubuntu server configs (netplan, BIND9, Kea, Nginx, UFW, rsyslog, SNMPv3) and setup script
b726a3b	03:20	Ansible playbooks, Python health check with tests, Packet Tracer configs
68d0bf5 · 2e95835	03:23–03:25	Test runner, SNMPv3 agents, troubleshooting runner, topology diagram, workbook
0b01bb0 · d8328d7	03:30–03:49	Packet Tracer build guide, monitoring docs, limitations, CV entry
eeaca00	04:04	Full test plan 15/15, troubleshooting 10/10, fixes
c9843bd	04:33	Topology built and verified in Packet Tracer 9.0
9f8a988	17:23	LibreNMS (SNMPv3) and WireGuard VPN, 17/17
3b6c0cb	18:34	Zabbix, NetBox, Grafana, Nmap audit, packet analysis, 22/22

Step 1: Read the guide and plan the repository

The 14-page lab guide was read and mapped to a repository layout (guide section 12): a folder for each layer, evidence next to each claim, and no secrets in Git. Tools: **Git**, **GitHub CLI**.

```
git init && git branch -M main
gh repo create muttakin210/Enterprise-Network-Infrastructure-Cisco-Linux --public --source . --push
# .gitignore keeps out: the PDF guide, .lab-local/ (lab passwords), vault.yml, raw backups, *.bin images
```

Step 2: Addressing plan and topology diagram

The VLAN/subnet plan was written as CSV, then turned into an Excel workbook and a PNG diagram by a Python script (`topology/make_topology_assets.py`). Tools: **Python** (openpyxl, Pillow).

```
python topology/make_topology_assets.py   # -> addressing-plan.xlsx + enterprise-topology.png
```

Step 3: Cisco IOS configurations

Configs for **HQ-R1, BR-R2** (IOSv / ISR) and **HQ-SW1, BR-SW1** (IOSvL2). Every secret is a `<PLACEHOLDER>`. Main parts of HQ-R1:

```
interface GigabitEthernet0/0.10
 description VLAN10-ADMIN
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
 ip helper-address 192.168.50.10          ! DHCP relay to Kea on srv01
interface GigabitEthernet0/0.99
 encapsulation dot1Q 99
 ip address 192.168.99.1 255.255.255.0
 ip access-group MGMT-PROTECT out          ! protect switch management
interface GigabitEthernet0/1
 ip address 10.0.12.1 255.255.255.252
 ip ospf network point-to-point
router ospf 1
 router-id 1.1.1.1
 passive-interface default
 no passive-interface GigabitEthernet0/1
 network 10.0.12.0 0.0.0.3 area 0
 network 192.168.10.0 0.0.0.255 area 0     ! ... 20, 50, 99
ip access-list extended MGMT-PROTECT
 permit ip 192.168.10.0 0.0.0.255 192.168.99.0 0.0.0.255
 permit ip host 192.168.50.10 192.168.99.0 0.0.0.255
 permit ip 192.168.199.0 0.0.0.255 192.168.99.0 0.0.0.255
 deny   ip any any log
ip ssh version 2
line vty 0 4
 login local
 transport input ssh
 access-class VTY-ADMIN in
logging host 192.168.50.10
snmp-server group NMS v3 priv access SNMP-NMS
snmp-server user monitor NMS v3 auth sha <SNMP_AUTH_SECRET> priv aes 128 <SNMP_PRIV_SECRET>
```

Switch hardening (HQ-SW1):

```
vlan 10 / 20 / 50 / 99 / 999
spanning-tree mode rapid-pvst
spanning-tree portfast bpduguard default
ip dhcp snooping vlan 10,20
interface GigabitEthernet0/0               ! uplink trunk
 switchport mode trunk
 switchport trunk native vlan 999
 switchport trunk allowed vlan 10,20,50,99
 switchport nonegotiate
 ip dhcp snooping trust
interface GigabitEthernet0/1               ! ADMIN-PC access port
 switchport mode access
 switchport access vlan 10
 switchport port-security maximum 2
 switchport port-security mac-address sticky
 switchport port-security violation restrict
 spanning-tree portfast
 ip dhcp snooping limit rate 10
```

On BR-R2, **FINANCE-PROTECT** sits outbound on G0/0.40. It permits Admin and srv01, denies IT and HR, and logs the denies.

Step 4: Ubuntu Server VM

A VirtualBox VM `ENL-Ubuntu-Server` was created with an unattended Ubuntu 22.04.5 install (2 vCPU, 2 GB RAM, later 3 GB; 30 GB disk). Host-only networking was broken on this PC, so **NAT port forwards bound to 127.0.0.1** are used, plus an internal network `enl-vlan50` for the lab NIC. Tools: **VirtualBox / VBoxManage**.

```
VBoxManage createvm --name ENL-Ubuntu-Server --ostype Ubuntu_64 --register
VBoxManage modifyvm ENL-Ubuntu-Server --memory 3072 --cpus 2 --nic1 nat --nic2 intnet --intnet2 enl-
vlan50
VBoxManage modifyvm ENL-Ubuntu-Server --natpf1 "ssh,tcp,127.0.0.1,2222,,22"     # also 8080 http, 8000
LibreNMS,
                                                                                 # 8081 Zabbix, 8082 NetBox,
3000 Grafana
VBoxManage unattended install ENL-Ubuntu-Server --iso ubuntu-22.04.5-live-server-amd64.iso --user
netadmin ...
ssh -p 2222 netadmin@127.0.0.1
```

Step 5: Linux network services with one script

`linux/setup-server.sh` builds srv01 from the repository and can be re-run safely. In order, it:

1. installs the packages;
2. generates lab secrets into `/root/lab-secrets.env` (mode 600);
3. applies netplan (bridge `br-vlan50` 192.168.50.10/24, routes to the lab via 192.168.50.1);
4. configures **Nginx**, **BIND9** (zone `corp.lab` and reverse zone, checked with `named-checkzone`), **Kea** DHCPv4 (checked with `kea-dhcp4 -t`), **rsyslog** (one log file per device) and the **SNMPv3** agent;
5. hardens SSH (no root login, MaxAuthTries 3, AllowUsers netadmin);
6. sets kernel options (`ip_forward=0`, rp_filter);
7. creates the automation SSH key;
8. applies the **UFW** policy.

```
sudo ./linux/setup-server.sh
named-checkzone corp.lab /etc/bind/zones/db.corp.lab
kea-dhcp4 -t /etc/kea/kea-dhcp4.conf
dig @127.0.0.1 www.corp.lab +short          # -> srv01 -> 192.168.50.10
sudo ufw status verbose                     # default deny incoming, per-service allow rules
```

Step 6: Emulating the Cisco topology on Linux

Licensed IOSv images were not available, and the guide forbids unofficial ones. So `testing/emulation/netns-lab.sh` rebuilds the same design on the VM:

- each router, switch and PC is a **network namespace**;
- switches are **VLAN-aware Linux bridges** with trunk and access ports;
- routers use **802.1Q subinterfaces**;
- OSPF runs in **FRR** (Cisco-like `vtysh`);
- DHCP relay is **isc-dhcp-relay**;
- the Cisco ACLs are mirrored rule for rule in **nftables**;
- each device runs **sshd** and **snmpd** v3.

srv01's bridge attaches to HQ-SW1's VLAN 50 port, just as the real server plugs into an access port.

```
sudo ./testing/emulation/netns-lab.sh up       # build everything; 'status' and 'down' also exist
sudo vtysh -N hq-r1 -c 'show ip ospf neighbor'   # 2.2.2.2  Full
sudo ip netns exec it-pc ping 192.168.40.100     # blocked by FINANCE-PROTECT (nftables)
```

Step 7: Automation with Ansible and Python

Ansible: the inventory has a `cisco_ios` group whose credentials go in an encrypted `vault.yml` (only an example file is committed). `backup.yml` saves each running config with `cisco.ios.ios_config backup: true`, then writes a **sanitized** copy with secrets replaced. `verify-ospf.yml` asserts that the neighbor is Full. An emulation variant runs the same logic against the namespaces.

Python: `health_check.py` reads `devices.csv`, pings and checks TCP ports in parallel threads, appends to a CSV log, prints a summary, and exits non-zero if anything is down, which makes it usable from cron or CI. `test_health_check.py` holds the unit tests.

```
ansible-playbook playbooks/backup.yml --ask-vault-pass
ansible-playbook playbooks/verify-ospf.yml
python3 automation/python/health_check.py -i devices.csv -o results/health_log.csv --port 22
python3 -m pytest automation/python -q
```

```
DEVICE      IP              STATUS        RTT ms  tcp/22
HQ-R1       192.168.99.1    reachable       0.10  open
BR-SW1      192.168.199.2   reachable       0.18  open
SRV01       192.168.50.10   reachable       0.56  open
6/6 devices reachable
```

Step 8: Test plan and troubleshooting challenges

`testing/run-tests.sh` runs the guide's section 10 test plan against the emulation. It saves the **raw output of every command** as evidence, writes `test-report.md` and redacts every lab secret. The first run gave 9/15. The fixes (UFW for relayed DHCP, client DNS, SNMP MIBs, backup folders, `pipefail`) brought it to **15/15**. `run-challenges.sh` injects 10 faults, records the diagnosis and verifies each fix.

```
sudo ./testing/run-tests.sh                       # -> testing/results/test-report.md + evidence/T*.txt
sudo ./testing/troubleshooting/run-challenges.sh  # -> TS-01..TS-10.md
```

Step 9: Packet Tracer build

`make_pt_configs.py` converts the IOSv configs to Packet Tracer syntax (2911 / 2960-24TT port names; PT does not support some keywords). The topology was built in **Packet Tracer 9.0**: 9 devices, generated configs pasted in, and SRV01 services (DHCP pools, DNS, HTTP, Syslog). Verified there:

- DHCP through the relay;
- OSPF routes and the WAN bounce;
- both ACLs;
- DNS and web;
- Syslog;
- VLANs and trunks;
- port security, DHCP snooping, BPDU Guard and Rapid-PVST on both switches.

There are 14 screenshots.

```
python cisco/packet-tracer/make_pt_configs.py
! verification in PT CLI
show ip ospf neighbor · show ip route ospf · show access-lists · show vlan brief · show interfaces trunk
show port-security · show ip dhcp snooping · show spanning-tree summary
```

Step 10: LibreNMS

`monitoring/librenms/install-librenms.sh` follows the official install:

- PHP **8.4** from ppa:ondrej/php (current LibreNMS requires it);
- MariaDB, Composer and a dedicated PHP-FPM pool;
- Nginx on port 8000;
- generating the app key, migrating the schema and adding the admin user;
- scheduler, cron and UFW rules;
- adding all 5 devices over SNMPv3 authPriv.

The result: 5/5 up, 31 ports discovered, and a WAN down/up event logged.

```
sudo ./monitoring/librenms/install-librenms.sh
sudo -u librenms lnms device:add --v3 --security-name=monitor --auth-protocol=SHA --auth-
password="$SNMP_AUTH" \
     --privacy-protocol=AES --privacy-password="$SNMP_PRIV" --port-association-mode=ifName 192.168.99.1
sudo -u librenms lnms device:poll all
sudo -u librenms /opt/librenms/validate.php
```

Step 11: WireGuard remote-admin VPN

`testing/emulation/wireguard-lab.sh` adds an internet link (203.0.113.0/30) and a `remote-admin` laptop namespace, and terminates the tunnel on HQ-R1:

- `wg0` 10.99.0.1/24 on UDP 51820;
- the client's `AllowedIPs` list only servers and the two management networks;
- on HQ-R1, **EDGE-IN** accepts only UDP 51820 from the internet, and **VPN-FILTER** drops anything from the tunnel to other networks, so Finance stays blocked even if the client widens its own `AllowedIPs`;
- OSPF advertises the VPN pool as a passive network, so BR-R2 can route replies back.

```
sudo ./testing/emulation/wireguard-lab.sh up
sudo ip netns exec hq-r1 wg show wg0          # latest handshake: 1 second ago
sudo ip netns exec remote-admin ping 192.168.50.10   # allowed through the tunnel
sudo ip netns exec remote-admin ping 192.168.40.100  # dropped (not routed / VPN-FILTER)
```

Step 12: Zabbix 8.0 LTS

`install-zabbix.sh` installs from the official repository: server, frontend on port 8081, and agent 2. It reuses PHP 8.4 and MariaDB and keeps caches small. `zabbix-configure.py` then configures everything through the JSON-RPC API:

- changes the default Admin password;
- stores the SNMPv3 keys as **secret macros**;
- creates the host groups;
- adds 4 SNMPv3 hosts (template "Linux by SNMP") and srv01 with the agent;
- runs "check now" on every discovery rule;
- creates a read-only API role and user for Grafana.

```
sudo ./monitoring/zabbix/install-zabbix.sh
python3 monitoring/zabbix/zabbix-check.py hosts
python3 monitoring/zabbix/zabbix-check.py poll hq-r1 'g0-1\(.*\): Operational status'
python3 monitoring/zabbix/zabbix-check.py problem hq-r1 g0-1 --timeout 150   # Link down opens, then
resolves
```

Step 13: NetBox 4.7 (source of truth)

`install-netbox.sh` installs NetBox 4.7.4 with Python 3.12 (deadsnakes) and PostgreSQL 17 (PGDG), because both are newer than Ubuntu 22.04 ships. It also sets up Redis, gunicorn with 2 workers, systemd units and Nginx on port 8082. `import-lab.py` then loads through the REST API (idempotent):

- sites, VLANs, prefixes and DHCP ranges, from the addressing CSV;
- device types and roles, and the 5 devices;
- 21 interfaces, with 802.1Q modes and parent subinterfaces;
- 12 IPs, with primary IPs and DNS names;
- 4 cables.

```
sudo ./ipam/netbox/install-netbox.sh
NETBOX_ADMIN_PASS=... python3 ipam/netbox/import-lab.py topology/addressing-plan.csv
python3 ipam/netbox/nb-expected.py          # documented IPs + DHCP ranges (used by Nmap audit)
```

Step 14: Grafana

`install-grafana.sh` installs Grafana OSS and the Zabbix plugin. It provisions two data sources:

- **Zabbix**, through the read-only API user;
- **LibreNMS**, through a MariaDB user with SELECT on three tables only.

It also provisions the dashboard "ENL Lab overview", disables sign-up and anonymous access, and opens port 3000 only to admins.

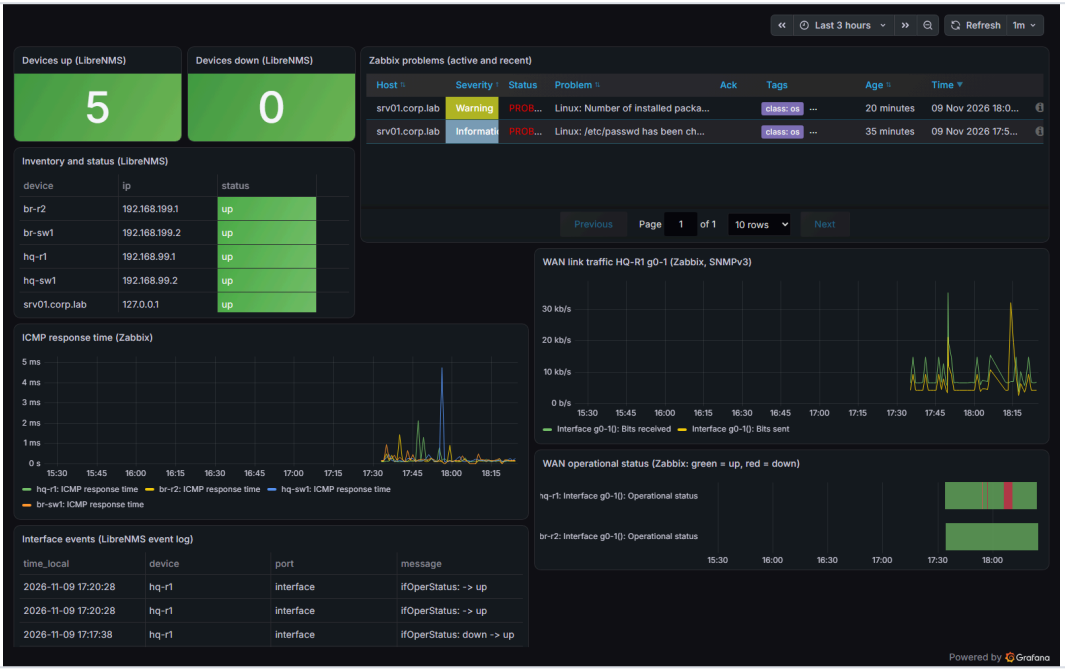


Figure 2. One dashboard. LibreNMS: inventory and interface events. Zabbix: problems, latency, WAN traffic and status. The red bands are the WAN failure tests.

Step 15: Nmap audit

`testing/nmap/nmap-audit.sh` scans from inside the lab:

1. a ping sweep of all 8 subnets, compared with NetBox: 16 live hosts, **0 undocumented**;
2. srv01's ports from Admin vs IT: the management UIs are open only from Admin;
3. Finance and switch management from Admin vs IT;
4. the internet edge: no open TCP, and SNMP silently dropped;
5. the SNMP agents: v3 only, and v2c "public" refused.

```
sudo ip netns exec admin-pc nmap -n -sn 192.168.10.0/24 192.168.20.0/24 ... 10.0.12.0/30
sudo ip netns exec it-pc nmap -n -Pn -sT -p 22,53,80,3000,8000,8081,8082 192.168.50.10
sudo ip netns exec remote-admin nmap -n -Pn -sU -p 51820,161,53 203.0.113.1
sudo ./testing/nmap/nmap-audit.sh            # exit code = failed checks (0)
```

Step 16: Packet analysis with tcpdump, tshark and Wireshark

`testing/capture/capture-lab.sh` triggers six real events and captures each where it is visible. It saves a pcap and a tshark summary for each. **Wireshark** on Windows opens the same files.

Capture	Finding
OSPF after a WAN flap	Hello → DB Description → LS Request → **LS Update** → LS Ack between 1.1.1.1 and 2.2.2.2
DHCP relay	unicast from the relay to Kea with **giaddr 192.168.20.1**, which selects the IT pool
SNMPv3	user "monitor" visible, **encryptedPDU**, no community string; v2c "public" gets no reply
WireGuard outside	only UDP 51820 "transport data"; **no ICMP visible**
WireGuard inside (wg0)	the admin's ICMP in clear after decryption
Syslog	readable on the wire, which is a known weakness (use TLS/RELP or a management VLAN)

```
sudo ip netns exec hq-r1 tcpdump -i g0-1 -nn -w ospf.pcap 'proto ospf'
tshark -r ospf.pcap -T fields -e ip.src -e ospf.msg -e ospf.srcrouter
tshark -r snmp.pcap -Y 'snmp.msgVersion == 3 && snmp.encryptedPDU'
wireshark -r snmp.pcap -Y snmp                # Windows GUI
```

Step 17: Evidence, screenshots and publishing

- The UI screenshots are scripted with **Playwright** (headless Chromium), and the Wireshark ones with PowerShell.
- Before every commit, all staged files (pcaps included) are scanned for every value in the lab secrets file and the WireGuard keys: **0 hits**.
- Commits carry clear messages.

```
python testing/screenshots/capture-ui.py .lab-local/ui-admin.txt
powershell -File testing/screenshots/capture-wireshark.ps1
git add -A && git diff --cached --name-only | xargs grep -lFf secrets.txt   # must print nothing
git commit -m "..." && git push origin main
```

4. Tool reference: what each tool is and does

For every tool: **What it is** · **What it can do** (in general) · **What I did with it** in this project.

4.1 Cisco and network tools

Cisco IOS `network OS`

What it is: the operating system of Cisco routers and switches, configured from a CLI.

What it can do:

- routing (static, OSPF, EIGRP, BGP) and switching (VLANs, STP, EtherChannel);
- ACLs, NAT, VPNs, QoS and first-hop redundancy;
- management features: SSH, SNMP, Syslog, NTP.

What I did:

- wrote full configs for 2 routers and 2 switches: router-on-a-stick, OSPF, DHCP relay, extended and standard ACLs, SSH v2 only;
- added login blocking, SNMPv3 authPriv, Syslog, port security, DHCP snooping, BPDU Guard and Rapid-PVST;
- replaced every secret with a placeholder.

Cisco Packet Tracer 9.0 `network simulator`

What it is: Cisco's free network simulator for learning and labs.

What it can do: simulate routers, switches, PCs and servers with a real-feeling IOS CLI; step through packets in simulation mode; run DHCP, DNS, HTTP and Syslog services on server icons.

What I did: built the full 9-device topology from generated configs and set up the SRV01 services. Then I verified relay DHCP, OSPF, the ACLs, DNS, web, Syslog and the switch security features, with 14 screenshots. Its limits: no SNMPv3, and it cannot connect to real VMs.

FRR (FRRouting) `routing suite`

What it is: an open-source routing suite for Linux (OSPF, BGP, IS-IS, ...) with a Cisco-like shell, `vtysh`.

What it can do: turn a Linux box into a router; used in data centers, labs and SONiC switches.

What I did: ran OSPF area 0 in the HQ-R1 and BR-R2 namespaces (router IDs 1.1.1.1 / 2.2.2.2, point-to-point WAN, passive LANs). I used it to test adjacency, routes, WAN failure (11 s re-convergence) and the troubleshooting faults (hello mismatch, missing network statement).

Linux network namespaces, bridges and veth `kernel networking`

What it is: kernel features that give a process its own isolated network stack, plus virtual switches (bridges) and virtual cables (veth pairs).

What it can do: they are the building blocks of containers and Kubernetes networking, and they make lightweight network emulators possible.

What I did: one namespace per router, switch and PC, VLAN-aware bridges as switches with trunk and access ports, and veth pairs as cables.

nftables Linux firewall

What it is: the modern Linux packet-filtering framework, the successor to iptables.

What it can do: stateful and stateless filtering, NAT, sets and maps, logging and counters.

What I did: mirrored the Cisco ACLs rule for rule (MGMT-PROTECT, FINANCE-PROTECT) with counters, and built the WireGuard EDGE-IN and VPN-FILTER policy on HQ-R1.

isc-dhcp-relay (dhcrelay) DHCP relay

What it is: the Linux equivalent of `ip helper-address`.

What it can do: forward DHCP broadcasts as unicast to a central server, adding the giaddr.

What I did: relayed every client VLAN to Kea. This also uncovered a real bug: BR-R2 relays from its WAN address, so UFW had to allow 10.0.12.0/30.

WireGuard VPN

What it is: a modern, minimal VPN in the Linux kernel. It uses public-key peers, UDP, and ChaCha20-Poly1305 encryption.

What it can do: remote-access and site-to-site VPNs with very little configuration and high performance.

What I did:

- built a remote-admin VPN on HQ-R1 with least-privilege `AllowedIPs`;
- added an edge filter (only UDP 51820) and a router-side VPN-FILTER;
- advertised the pool in OSPF;
- tested the handshake, the permitted and denied routes, and an attack where the client widens its own AllowedIPs (still blocked).

4.2 Linux server tools

VirtualBox hypervisor

What it is: a free type-2 hypervisor (VMware Workstation is the alternative).

What it can do: run VMs with NAT, bridged, host-only and internal networking; snapshots; and full control from the CLI with VBoxManage.

What I did: created the Ubuntu VM with an unattended install. When the host-only driver was missing, I worked around it with 127.0.0.1-bound NAT port forwards and an internal network. Later I added forwards for each web UI.

Ubuntu Server 22.04 + Netplan OS / network config

What it is: a long-term-support Linux server OS. Netplan is its YAML network configuration layer.

What it can do: host any service. Netplan declares interfaces, bridges, VLANs, routes and DNS.

What I did: a bridge `br-vlan50` with a static IP 192.168.50.10, lab routes via HQ-R1, and the server resolving through its own BIND.

BIND9 DNS

What it is: the reference authoritative and recursive DNS server.

What it can do: authoritative zones, recursion, views, DNSSEC and zone transfers.

What I did:

- zone `corp.lab`: servers, devices and gateways, with CNAMEs www, intranet, syslog and nms;
- the reverse zone `168.192.in-addr.arpa`;
- recursion only for lab networks;
- validated with `named-checkconf` and `named-checkzone`.

Kea DHCPv4 DHCP

What it is: ISC's modern DHCP server, with JSON config and an API. It replaced isc-dhcp-server.

What it can do: multiple subnets and pools, reservations, high availability and lease databases.

What I did: one subnet per client VLAN (pools .100–.199, router option, DNS 192.168.50.10, domain corp.lab). Kea picks the subnet by the relay giaddr.

Nginx web server / reverse proxy

What it is: a high-performance web server and reverse proxy.

What it can do: static sites, reverse proxying to apps, TLS termination and load balancing.

What I did:

- the intranet site on port 80 with `server_tokens off`;
- LibreNMS on 8000 and Zabbix on 8081, both through PHP-FPM;
- a reverse proxy for NetBox on 8082 (to gunicorn).

rsyslog central logging

What it is: the standard Linux syslog daemon.

What it can do: receive logs over UDP/TCP/TLS, filter them, write templated files, and forward to a SIEM.

What I did: a dedicated "remote" ruleset on UDP and TCP 514 that writes one file per device plus `all-devices.log`. It recorded the OSPF `Full→Deleted→Full` sequence during the WAN test.

Net-SNMP SNMP agent & tools

What it is: SNMP agent (snmpd) and client tools (snmpget, snmpwalk).

What it can do: expose and query device metrics: interfaces, CPU, disks, processes.

What I did: SNMPv3 user `monitor` (authPriv, SHA/AES) on srv01 and on every emulated device. Tests prove that v2c and wrong keys are refused.

UFW and OpenSSH hardening host security

What it is: UFW is a simple front end to the Linux firewall. OpenSSH is the SSH server.

What it can do: per-port and per-source allow rules with a default deny; key-based, restricted remote login.

What I did:

- default deny incoming;
- each service is allowed only from the sources that need it, for example the web UIs only from the Admin VLAN, VPN admins and the host;
- Syslog and traps only from device addresses;
- SSH: no root login, 3 tries, AllowUsers netadmin.

4.3 Automation

Ansible configuration automation

What it is: agentless automation over SSH using YAML playbooks. The `cisco.ios` collection drives IOS.

What it can do: config backups, pushing standard configs, compliance checks, upgrades, and fleet-wide show commands.

What I did:

- inventory with group_vars, and credentials in Ansible Vault;
- `backup.yml`: config backups with automatic sanitizing;
- `verify-ospf.yml`: asserts the OSPF neighbor is Full;
- an emulation variant that is exercised in T12.

Python scripting

What it is: the main scripting language of network automation (Netmiko, NAPALM, Nornir, requests).

What it can do: health checks, API integrations, report generation and data processing.

What I did:

- `health_check.py`: threaded ping and TCP checks with a CSV log and unit tests;
- API clients for Zabbix and NetBox (standard library only, idempotent);
- the Packet Tracer config converter;
- topology and workbook generation;
- Playwright screenshots.

4.4 Monitoring, IPAM and analysis

LibreNMS network monitoring (NMS)

What it is: an open-source, auto-discovering network monitoring system (PHP/Laravel, MariaDB, RRD).

What it can do: discover devices and ports over SNMP, graph traffic, errors, CPU and memory, keep an event log, send alerts, and run billing and weathermaps.

What I did: installed it by script, added all devices over SNMPv3, discovered 31 ports, and logged the WAN `ifOperStatus up→down→up` event. Its database also feeds Grafana.

Zabbix enterprise monitoring

What it is: an enterprise monitoring platform: server, agents, SNMP, templates, triggers, actions and an API.

What it can do: monitor networks, servers, apps and the cloud; collect with SNMP, agents, HTTP and scripts; fire triggers that escalate and notify; discover with LLD; manage everything through the API.

What I did:

- configured it entirely through the API: secret macros for the SNMPv3 keys, host groups, templates, the agent on srv01, and a read-only Grafana role;
- showed that a WAN carrier loss opens a "Link down" problem and resolves it.

Grafana visualization

What it is: a dashboard and visualization platform.

What it can do: connect to many data sources (Prometheus, SQL, Zabbix, Loki…), build dashboards and alerts, and provision them as code.

What I did: one provisioned dashboard combining Zabbix and LibreNMS, with read-only accounts for both and anonymous access disabled.

NetBox IPAM / DCIM source of truth

What it is: the standard open-source tool for documenting networks (IP addresses, VLANs, devices, cables, racks), built on Django and PostgreSQL.

What it can do: act as the "intended state" that automation reads from; IPAM; cabling and rack documentation; a REST and GraphQL API; webhooks.

What I did: loaded the whole design through the API (idempotent), then used it as the reference for the Nmap audit: *documentation vs reality*.

Nmap discovery & security scanning

What it is: the standard network scanner.

What it can do: host discovery, port scanning (TCP, UDP), service and version detection, OS detection, and NSE scripts.

What I did: an audit that proves:

- there are no undocumented hosts;
- UFW shows different ports per VLAN;
- the ACLs work;
- the internet edge exposes nothing;
- the SNMP agents accept only v3.

tcpdump, tshark and Wireshark packet analysis

What they are: tcpdump captures packets on the CLI. tshark is Wireshark's CLI decoder. Wireshark is the GUI analyzer.

What they can do: see exactly what is on the wire, troubleshoot protocols, verify encryption, and measure timing.

What I did: captured OSPF adjacency building, DHCP relay with giaddr, SNMPv3 encryption, WireGuard inside and outside the tunnel, and plaintext Syslog. The checks were automated with tshark filters and the captures viewed in Wireshark.

4.5 Supporting tools

Tool	Used for
Git / GitHub / gh	Version control, a public portfolio repo and staged commits
MariaDB / PostgreSQL / Redis	Databases for LibreNMS and Zabbix (MariaDB), NetBox (PostgreSQL) and the NetBox task queue and cache (Redis)
PHP-FPM / gunicorn	Application servers behind Nginx (PHP apps / Django)
systemd	Services and timers (librenms-scheduler, netbox, netbox-rq)
Playwright	Scripted, repeatable screenshots of the web UIs
PowerShell / Bash	Windows-host automation (VBoxManage, Wireshark capture) and all Linux scripts
openpyxl / Pillow	Excel addressing workbook and topology image

5. Command cheat sheet

Cisco IOS: configure

```
configure terminal
vlan 10 / name ADMIN
interface g0/0.10 / encapsulation dot1Q 10 / ip address 192.168.10.1 255.255.255.0 / ip helper-address
192.168.50.10
interface g0/1 / switchport mode trunk / switchport trunk native vlan 999 / switchport trunk allowed vlan
10,20,50,99
interface f0/1 / switchport mode access / switchport access vlan 10 / switchport port-security / spanning-
tree portfast
router ospf 1 / router-id 1.1.1.1 / passive-interface default / no passive-interface g0/1 / network
10.0.12.0 0.0.0.3 area 0
ip access-list extended FINANCE-PROTECT / permit ip 192.168.10.0 0.0.0.255 192.168.40.0 0.0.0.255 / deny ip
any any log
crypto key generate rsa modulus 2048 / ip ssh version 2 / line vty 0 4 / transport input ssh / login local
copy running-config startup-config      (or: write memory)
```

Cisco IOS: verify and troubleshoot

```
show ip interface brief          show running-config | section ospf
show vlan brief                  show interfaces trunk           show interfaces g0/1 switchport
show ip ospf neighbor            show ip ospf interface brief    show ip route ospf
show access-lists                show ip dhcp snooping binding   show port-security interface f0/1
show spanning-tree summary       show logging                    show snmp user
ping 192.168.50.10 source g0/0.10        traceroute 192.168.40.100        debug ip ospf adj (lab only)
```

Linux networking

```
ip -br addr / ip -br link / ip route / ip neigh       ss -tulpn (listening sockets)
bridge vlan show                                      ip netns list ; ip netns exec it-pc ping 192.168.40.1
sudo vtysh -N hq-r1 -c 'show ip ospf neighbor'         sudo nft list ruleset
dig @192.168.50.10 www.corp.lab +short                dig -x 192.168.50.10
sudo ufw status numbered                              sudo journalctl -u kea-dhcp4-server -f
tail -f /var/log/remote/all-devices.log               logger -n 192.168.50.10 -P 514 -d "test"
snmpwalk -v3 -l authPriv -u monitor -a SHA -A "$SNMP_AUTH" -x AES -X "$SNMP_PRIV" 192.168.99.1 IF-
MIB::ifOperStatus
```

Services and tools

```
systemctl status nginx named kea-dhcp4-server rsyslog snmpd zabbix-server grafana-server netbox
sudo -u librenms lnms device:add / device:poll all / ./validate.php
wg show ; wg-quick up lab
ansible-playbook playbooks/backup.yml --ask-vault-pass ; ansible-vault create vault.yml
nmap -sn 192.168.10.0/24 ; nmap -sT -p 22,80 host ; nmap -sU -p 161 host ; nmap -sV host
tcpdump -i eth0 -nn -w cap.pcap 'udp port 67 or udp port 68'
tshark -r cap.pcap -Y dhcp -T fields -e dhcp.ip.relay
curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8081/
```

Wireshark display filters used

```
ospf        ospf.msg == 4 (LS Update)        dhcp        dhcp.ip.relay == 192.168.20.1
snmp        snmp.msgVersion == 3 && snmp.encryptedPDU          wg       wg.type == 4 (transport data)
icmp        syslog contains "LINEPROTO"       ip.addr == 192.168.50.10       tcp.port == 22
```

6. Rebuild the lab from zero

```
# on Windows (host)
git clone https://github.com/muttakin210/Enterprise-Network-Infrastructure-Cisco-Linux.git
# create the Ubuntu 22.04 VM (3 GB RAM, NAT forwards 2222/8080/8000/8081/8082/3000), copy the repo in, then
on the VM:
sudo ./linux/setup-server.sh                       # 1. server services + UFW
sudo ./testing/emulation/netns-lab.sh up           # 2. emulated Cisco topology (OSPF, VLANs, ACLs)
sudo ./testing/emulation/wireguard-lab.sh up       # 3. remote-admin VPN
sudo ./monitoring/librenms/install-librenms.sh     # 4. LibreNMS         http://127.0.0.1:8000
sudo ./monitoring/zabbix/install-zabbix.sh         # 5. Zabbix           http://127.0.0.1:8081
sudo ./ipam/netbox/install-netbox.sh               # 6. NetBox           http://127.0.0.1:8082
sudo ./monitoring/grafana/install-grafana.sh       # 7. Grafana          http://127.0.0.1:3000
sudo ./testing/run-tests.sh                        # 8. T01-T22 -> testing/results/test-report.md
sudo ./testing/troubleshooting/run-challenges.sh   # 9. TS-01..TS-10
# Packet Tracer: open cisco/packet-tracer/enterprise-lab.pkt (or build from configs/ with the README steps)
```

> Every installer is **idempotent** (safe to re-run) and generates its passwords into `/root/lab-secrets.env` (mode 600). Nothing secret is ever committed.

7. Test results and troubleshooting challenges

ID	Test	What is checked	Result
T01	Basic reachability	every client pings its gateway	PASS
T02	DHCP via relay	clients lease from their own VLAN pool through the relay	PASS
T03	OSPF	neighbor Full, remote LANs installed	PASS
T04	Inter-VLAN and WAN	permitted flows pass; traceroute path across the WAN	PASS
T05	Web	HTTP 200 for www.corp.lab from every VLAN	PASS
T06	DNS	forward, reverse and recursive answers	PASS
T07	FINANCE-PROTECT	IT/HR blocked (counters rise), Admin allowed	PASS
T08	MGMT-PROTECT	only Admin and srv01 reach the management SVIs	PASS
T09	WAN failure	adjacency drops, Syslog event, re-converges (11 s)	PASS
T10	Central Syslog	per-device files with source IPs	PASS
T11	SNMPv3	authPriv answers; v2c and wrong keys refused	PASS
T12	Ansible backup	backups created, OSPF asserted Full	PASS
T13	Python health check	all devices reachable, CSV written	PASS
T14	Server hardening	UFW default deny, root SSH off	PASS
T15	Linux services	all services active	PASS
T16	WireGuard	handshake, permitted routes, Finance blocked even with widened AllowedIPs	PASS
T17	LibreNMS	UI 200, 5 devices up via v3, port events	PASS
T18	Zabbix	5 hosts available; Link down opens and resolves	PASS
T19	NetBox	import correct and idempotent	PASS
T20	Grafana	both data sources OK, dashboard present, anonymous 401	PASS
T21	Nmap audit	0 undocumented hosts; policy as designed	PASS
T22	Packet analysis	OSPF LSUs, giaddr, SNMP encrypted, only UDP 51820 outside	PASS

Troubleshooting challenges (fault injected → diagnosed → fixed)

ID	Symptom	Root cause	How it was found
TS-01	PC cannot reach its gateway	access port in the wrong VLAN	`show vlan brief` / `bridge vlan show`
TS-02	VLAN 10 ↔ 20 fails	subinterface shut down	`show ip interface brief`
TS-03	Branch cannot reach HQ server	missing OSPF network statement	`show ip route` on BR-R2, then the HQ-R1 OSPF config
TS-04	OSPF neighbor does not form	hello/dead interval mismatch	`show ip ospf interface` on both ends
TS-05	VLAN missing across the trunk	VLAN pruned from the allowed list	`show interfaces trunk`
TS-06	DNS unreachable	no firewall rule for 53	`dig` timeout → `ufw status`

TS-07	No DHCP leases at the branch	missing ip helper-address / relay	tcpdump shows broadcasts that never reach Kea
TS-08	ACL blocks legitimate traffic	deny placed above the permit	`show access-lists` (counters, order)
TS-09	Ansible cannot connect	SSH not allowed on management	`nc -zv`, VTY ACL / sshd
TS-10	SNMP cannot see a device	SNMPv3 key mismatch	`snmpget` → "authorizationError / wrong digest"

> **Method used every time:** define the symptom → check bottom-up (L1 link → L2 VLAN/trunk → L3 IP/route → L4 ports/firewall → application) → compare with the documented design → change *one* thing → re-test and record the evidence.

8. Problems hit and how they were solved

These are real problems from the build, not invented ones. They make the best interview stories.

#	Problem	Root cause	Fix
1	VirtualBox host-only adapter could not be created	network driver missing on the host	NAT port forwards on 127.0.0.1 + internal network
2	SSH refused after reboot	the banner came from the installer; openssh-server was not installed	installed it from the console; the script installs it now
3	Branch clients got no DHCP	relay sends from the WAN IP 10.0.12.2; UFW allowed only 192.168.0.0/16	UFW rule for udp/67 from 10.0.12.0/30
4	Clients could not resolve names	dhclient handed DNS to systemd-resolved, which does not run in the namespace	write the leased DNS into the namespace's resolv.conf
5	Traceroute showed only *	UDP probes hit the UFW default deny	use ICMP traceroute (-I)
6	snmpget "Unknown Object Identifier"	Ubuntu ships without MIB files	snmp-mibs-downloader + enable MIBs
7	SNMP keys appeared in evidence	the test log echoed full command lines	redaction + a pre-commit scan for every secret
8	Ansible "destination does not exist"	backup folders were assumed to exist	task creates raw/ and sanitized/
9	Checks failed although the output looked right	`pipefail` + `grep -q` → SIGPIPE	removed pipefail from the test scripts
10	PT rejected config lines and ACLs lost their deny	PT's parser lacks some keywords; a rejected paste line is skipped silently	converter rewrites them; `show access-lists` after every paste
11	"Invalid DNS" in PT	value typed twice	clear the field, enter it once
12	LibreNMS composer: lock file incompatible	current LibreNMS needs PHP ≥ 8.4	PHP 8.4 from ppa:ondrej/php
13	"No application encryption key"	the web installer normally creates APP_KEY	`php artisan key:generate` in the script
14	`--seclevel` option does not exist	LibreNMS renamed the SNMPv3 flags	new flag names, found with `--help`
15	Unknown column 'snmpver'	credentials moved to an encrypted secrets table	test joins the new tables
16	Zabbix refused MariaDB 10.6	Zabbix 8.0 requires ≥ 10.11	documented `AllowUnsupportedDBVersions=1` (shared DB)
17	hostinterface.update unknown	not in the Zabbix 8.0 API	`host.update` with interfaceid
18	Grafana: no permission for hostgroup.get	Zabbix 8.0 disables API access in its default roles	dedicated read-only role (`*.get` only)
19	NetBox would not install on 22.04	needs Python ≥ 3.12 and PostgreSQL ≥ 15	deadsnakes Python 3.12 + PGDG PostgreSQL 17
20	Zabbix missed the WAN failure	admin shutdown made LLD treat the port as lost and disable its items; interface items are dependent items	simulate a far-end (carrier) failure; poll the master walk item

21	Nmap said `open|filtered`	UDP + silent drop gives no reply	corrected the expectation (that result *is* the drop)
22	Grafana problems panel empty	wrong panel id / query field in plugin 6.x	fixed the panel type and query
23	Events 6 h in the future	local time stored, Grafana assumed UTC	format the time as text in SQL

9. Security decisions

- **Management plane:**
  - SSH v2 only, with a local user and an enable secret;
  - a VTY ACL (Admin VLAN, the management VLANs and srv01);
  - login blocking, exec timeouts and a banner.
- **Segmentation:** VLANs per department; FINANCE-PROTECT and MGMT-PROTECT applied outbound on the protected subinterface, with deny logging.
- **Layer 2:**
  - unused ports shut and placed in VLAN 999;
  - native VLAN 999 and DTP off (`nonegotiate`);
  - port security (sticky, max 2, restrict);
  - DHCP snooping (trust only the uplink, rate-limit clients), which stops rogue DHCP servers;
  - BPDU Guard plus PortFast, which stops rogue switches and loops.
- **Monitoring:**
  - SNMPv3 authPriv only, with an ACL to the NMS;
  - keys stored as Zabbix secret macros and as encrypted secrets in LibreNMS;
  - Grafana uses read-only accounts.
- **Server:**
  - UFW default deny with per-source rules;
  - no root SSH and AllowUsers;
  - `ip_forward=0`;
  - `server_tokens off`.
- **Remote access:**
  - WireGuard on the edge router, with least-privilege AllowedIPs;
  - a router-side filter so the client cannot grant itself access;
  - the internet edge exposes only UDP 51820, which Nmap confirmed.
- **Repository:** no IOS images, passwords, keys, SNMP secrets or public IPs; placeholders, Ansible Vault and a pre-commit secret scan.
- **Known weakness, documented:** Syslog over UDP is plaintext. Fix with TLS/RELP or by keeping it on the management VLAN.

10. Limitations and honest scope

Area	Limitation	How it is handled
Cisco images	no licensed IOSv/IOSvL2, so no GNS3 with real IOS	IOS configs verified in Packet Tracer; end-to-end tests on a Linux emulation of the same design

Packet Tracer	no SNMPv3; cannot connect to VMs	SNMPv3 and monitoring tested in the emulation
Emulation	FRR/nftables behave like IOS but are not IOS; port security and DHCP snooping cannot be emulated	those were verified in Packet Tracer
Zabbix DB	MariaDB 10.6 below the 8.0 minimum	documented override; Ubuntu 24.04 for production
Memory	4 web platforms on a 3 GB VM	swap, small caches, 2 NetBox workers

> **In an interview, say this clearly:** "Configs are real IOS and were verified in Packet Tracer. Because I had no licensed images, I emulated the same design on Linux (FRR and nftables) to run automated end-to-end tests and monitoring." Honesty about scope builds trust.

11. Interview preparation

11.1 Elevator pitch (30 seconds)

> "I designed and built a two-site enterprise network lab. Cisco routers and switches use VLANs, router-on-a-stick and OSPF, secured with ACLs, SSH-only management, port security, DHCP snooping and SNMPv3. An Ubuntu server provides DNS, DHCP through relays, web, central Syslog and monitoring with LibreNMS, Zabbix and Grafana. NetBox is the source of truth. I automated backups with Ansible and health checks with Python. Then I proved everything with 22 automated tests, 10 troubleshooting scenarios and packet captures. I documented 23 real problems I solved along the way."

11.2 Two-minute walkthrough (structure)

1. **Goal:** a realistic small enterprise with HQ and a branch, built like a real job: design, build, secure, automate, monitor, prove.
2. **Design:** 7 VLANs, a /30 WAN, point-to-point OSPF, central DHCP and DNS on Linux, an access policy matrix.
3. **Security:** layered: management plane, segmentation ACLs, Layer-2 protections, SNMPv3, host firewall, a least-privilege VPN.
4. **Operations:** Ansible backups (sanitized), a Python health check, Syslog, two NMSs, a Grafana dashboard, NetBox, and an Nmap audit against NetBox.
5. **Proof:** 22 tests with raw evidence, a measured 11-second OSPF failover, Wireshark showing SNMPv3 and WireGuard encryption.
6. **Learning:** pick one story from section 8 (for example the DHCP relay from the WAN IP, or Zabbix losing the shut interface).

11.3 STAR stories (use these for behavioral questions)

Story 1: Branch DHCP failing (troubleshooting).
S: HQ clients got addresses but branch clients didn't. **T:** find out why before the test run.
A: I checked bottom-up. The relay was configured and Kea was running. tcpdump on the server showed *nothing* arriving from the branch. The relay forwards from its upstream interface, so the source was 10.0.12.2, and UFW only allowed udp/67 from 192.168.0.0/16.
R: One precise rule for 10.0.12.0/30 fixed it, and the lesson is in the build log: *check the source address a relay actually uses.*

Story 2: Monitoring that went blind (deep root cause).
S: The Zabbix WAN-failure test timed out, and Grafana showed "No data" for the WAN. **T:** make the alert reliable.
A: I found the interface items were *disabled*. Shutting the port set ifAdminStatus down, the discovery filter dropped it, and Zabbix 8 disables "lost" items. The interface items were also dependent items, so "check now" had to target the master SNMP walk.
R: The test now simulates a far-end carrier failure, which is more realistic. It polls the master item, and the problem opens in about 5 s and resolves on recovery. *Lesson: an admin shutdown and a real failure look different to monitoring.*

Story 3: Security that cannot be bypassed by the client (design).
S: Remote admins needed VPN access. **T:** least privilege. **A:** AllowedIPs limited to servers and management, *plus* a router-side VPN-FILTER and an edge filter allowing only UDP 51820. I then tested an attacker-style change: the client added Finance to its own AllowedIPs. **R:** It was still dropped at HQ-R1, which shows defense in depth works.

Story 4: Version drift (adaptability). LibreNMS needed PHP 8.4, NetBox needed Python 3.12 and PostgreSQL 15+, and Zabbix needed newer MariaDB. I read the official requirements, used the vendors' own repositories (ondrej PPA, deadsnakes, PGDG) and documented one consciously accepted trade-off (the Zabbix DB override) instead of breaking LibreNMS.

11.4 Technical questions and answers

Switching and VLANs

Q: What is a VLAN and why use it?

A: A logical Layer-2 broadcast domain. It separates departments for security and less broadcast traffic, and traffic between VLANs must go through a router or firewall, where policy is applied.

Q: Access port vs trunk port?

A: An access port carries one VLAN untagged (for end devices). A trunk carries many VLANs with 802.1Q tags (switch to switch or router). I limit trunks with `allowed vlan` and use an unused native VLAN 999.

Q: What is the native VLAN and why change it?

A: Frames on the native VLAN cross a trunk untagged. Leaving it as VLAN 1 with users on it allows VLAN-hopping by double tagging. I set it to an unused VLAN 999 and turned DTP off.

Q: What does DHCP snooping do?

A: It trusts DHCP server replies only on trusted ports (the uplink) and drops them elsewhere. That stops rogue DHCP servers. It also builds a binding table used by DAI and IP Source Guard. I rate-limited client ports.

Q: Port security: what and how?

A: It limits which and how many MACs can use a port. I used a maximum of 2, sticky learning and violation *restrict*: drop and log, without err-disabling the port.

Q: PortFast and BPDU Guard?

A: PortFast moves edge ports straight to forwarding. BPDU Guard err-disables a PortFast port if a BPDU arrives, which stops someone plugging in a switch or creating a loop.

Q: Why Rapid-PVST?

A: 802.1w converges in about a second instead of 30–50 s with classic STP, and it runs per VLAN.

Routing and OSPF

Q: How does router-on-a-stick work?

A: One physical interface with a subinterface per VLAN (`encapsulation dot1Q`). Each subinterface is the VLAN's gateway, and the router routes between them. Good for small sites; a Layer-3 switch scales better.

Q: Explain how an OSPF adjacency forms.

A: Down → Init → 2-Way (hellos) → ExStart/Exchange (DBD) → Loading (LSR/LSU) → Full. I saw exactly this in Wireshark: Hello, DB Description, LS Request, LS Update, LS Ack.

Q: What must match for neighbors to form?

A: Area, hello/dead timers, subnet and mask, authentication, the stub flag and MTU (for Exchange), plus a unique router ID. TS-04 was a hello/dead mismatch.

Q: Why ip ospf network point-to-point on the WAN?

A: There are only two routers on a /30, so no DR/BDR election is needed. Adjacency forms faster with no wait timer.

Q: What is passive-interface?

A: The network is still advertised but no hellos are sent there. I used `passive-interface default`, so OSPF only talks on the WAN. That is more secure and less noisy.

Q: How fast did your network fail over, and how did you measure it?

A: 11 seconds, measured by the test script from the link restore to the neighbor reaching Full and routes returning. Syslog recorded Full→Deleted and then Loading→Full. BFD or tuned timers would make it faster.

Q: OSPF cost and administrative distance?

A: Cost = reference bandwidth / interface bandwidth; lowest total wins. AD ranks protocols: connected 0, static 1, eBGP 20, OSPF 110, RIP 120.

ACLs and security

Q: Standard vs extended ACL, and where to place them?

A: A standard ACL matches source only, so place it near the destination. An extended ACL matches source, destination, protocol and ports, so it usually goes near the source. I put FINANCE-PROTECT outbound on the Finance subinterface, so one ACL protects the VLAN no matter where traffic comes from.

Q: Why does ACL order matter?

A: Rules are checked top-down and the first match wins, with an implicit deny at the end. TS-08 was a deny placed above a permit. `show access-lists` hit counters reveal it.

Q: How do you secure device management?

A: SSH v2 only, local or AAA users, an enable secret, a VTY access-class, a management VLAN, login blocking, timeouts, a banner, SNMPv3 instead of v2c, Syslog and NTP.

Q: SNMPv2c vs v3?

A: v2c uses a plaintext community string. v3 adds users with authentication (SHA) and encryption (AES), the authPriv level. My Wireshark capture shows encryptedPDU and no community string.

Q: What is defense in depth? Give an example from your project.

A: Multiple independent controls. VPN access is limited by the client's AllowedIPs, by a router VPN-FILTER, by an edge filter, by UFW on the server and by MGMT-PROTECT. If one is misconfigured, the others still hold.

Services: DHCP, DNS, Syslog

Q: Explain DORA and the role of a relay.

A: Discover, Offer, Request, Ack. Broadcasts don't cross routers, so the relay (`ip helper-address`) unicasts them to the server and adds the giaddr. The server uses the giaddr to pick the right subnet's pool.

Q: Forward vs reverse DNS zone? Why check zones?

A: Forward maps name→IP (A, CNAME). Reverse maps IP→name (PTR). `named-checkzone` catches syntax and serial mistakes before a reload breaks DNS.

Q: Why central Syslog?

A: Device logs are lost on reboot and are scattered. A central server gives correlation and retention, and it can feed a SIEM. I keep one file per device plus a combined log, and use timestamps with msec.

Linux and server

Q: How did you harden the Ubuntu server?

A: UFW default deny with per-source rules, no root SSH, MaxAuthTries 3, AllowUsers, `ip_forward=0`, rp_filter, only the needed services, generated secrets in a 600 file, and server_tokens off.

Q: What is a network namespace and why did you use them?

A: An isolated network stack. Each one acts like a separate device. I used them to emulate routers, switches and PCs on one VM so I could run real routed tests without licensed images. Containers use the same technology.

Q: Your script is "idempotent"; what does that mean?

A: Running it twice gives the same result with no duplicates or errors. It checks before it creates, for example "create the user if missing". That is essential for automation.

Automation

Q: Why Ansible for network devices?

A: It is agentless over SSH, declarative YAML, has vendor modules (cisco.ios) and is repeatable. I used it for backups and compliance (an OSPF Full assertion), with credentials in Ansible Vault.

Q: Why sanitize backups?

A: Configs contain password hashes and SNMP keys. The sanitized copy replaces them with placeholders, so it can be stored in Git safely.

Q: What does your Python script do and how is it tested?

A: It reads a CSV inventory, pings and checks TCP ports concurrently, logs to CSV, prints a summary, and returns a non-zero exit code on failure (for cron or CI). Unit tests cover parsing and logic.

Monitoring, NetBox, Nmap, Wireshark, VPN

Q: Why both LibreNMS and Zabbix?

A: To compare them. LibreNMS auto-discovers and is network-focused, with great port graphs. Zabbix is a broader enterprise platform with triggers, actions and an API. Grafana unifies both. In a real job I would pick one based on scope.

Q: How did you keep monitoring credentials safe?

A: SNMPv3 keys are secret macros in Zabbix and encrypted in LibreNMS. Grafana uses a Zabbix role allowed only `*.get` and a database user with SELECT on three tables.

Q: What is NetBox and how did you use it?

A: A source of truth for IPAM and DCIM. I loaded the design through its API, then audited reality against it with Nmap: 16 live hosts, 0 undocumented. Automation should read the intended state from NetBox.

Q: Nmap says open|filtered for UDP; what does it mean?

A: No response. A closed UDP port would send ICMP port-unreachable and an open service would reply, so no reply means it is filtered (silently dropped) or open but silent. For my edge test it proved the drop.

Q: How do you prove traffic is encrypted?

A: Capture it. Outside the WireGuard tunnel there is only UDP 51820 transport data and no ICMP. Inside wg0 the ICMP is visible. SNMPv3 shows encryptedPDU, while Syslog was readable, which is a documented weakness.

Q: WireGuard vs IPsec?

A: WireGuard is small, kernel-based and uses modern fixed crypto with simple key pairs. IPsec/IKEv2 is the industry standard on Cisco gear. On Cisco I would build the same policy with IKEv2 remote access plus a VPN filter ACL.

Troubleshooting scenarios

Q: A user can't reach the server. Walk me through it.

A: Scope it first: one user or many, and since when. Then go bottom-up:

1. link and port status;
2. VLAN and trunk (`show vlan`, `show interfaces trunk`);
3. IP, mask and gateway, then ping the gateway;
4. routing on both directions (`show ip route`);
5. ACLs and firewalls (`show access-lists`, `ufw`);
6. DNS vs IP: ping by IP first;
7. the service itself (`ss -tulpn`, logs).

Change one thing at a time, then verify.

Q: OSPF neighbor stuck in ExStart/Exchange?

A: Usually an MTU mismatch. Check `show ip ospf interface` on both sides and fix the MTU (or `ip ospf mtu-ignore` as a workaround).

Q: DHCP works at HQ but not the branch?

A: Check the helper-address on the branch subinterface, the route back to the relay address, and the firewall on the server for the relay's *actual* source IP. That last one was my real bug.

11.5 Questions you can ask the interviewer

- What does your network look like: sites, vendors, routing protocols?
- Which monitoring and IPAM tools does the team use, and is there a source of truth?
- How much is automated today (Ansible, Python), and what would you like automated next?
- How are changes and incidents handled: change windows, on-call, post-incident reviews?
- What would success look like in the first 90 days?

11.6 Tips for the interview

- Open the GitHub repo and the Grafana screenshot. **Showing** is stronger than telling.
- For every claim, have the evidence file ready: test-report.md, the build log and the pcaps.
- Be honest about scope (Packet Tracer plus emulation, not physical gear), then explain why the approach is still rigorous.
- Practice the CLI: `show ip ospf neighbor`, `show vlan brief`, `show access-lists`, `show interfaces trunk`.
- When you don't know something, say how you would find out: docs, `--help`, a lab test, packet capture. That is exactly how this project was built.

12. Glossary

Term	Meaning
802.1Q	VLAN tagging standard: a 4-byte tag carries the VLAN ID on trunks
ACL	Access Control List: ordered permit/deny rules, first match wins, implicit deny at the end
authPriv	SNMPv3 security level with authentication (SHA) and privacy/encryption (AES)
BPDU Guard	Err-disables an edge port that receives spanning-tree BPDUs
DCIM / IPAM	Data-center infrastructure management / IP address management (NetBox does both)
DHCP relay / giaddr	Forwards DHCP to a central server; giaddr = the relay's address, used to choose the pool
DHCP snooping	Switch feature that blocks DHCP server replies on untrusted ports
Idempotent	Can be run many times with the same end result
LLD	Zabbix low-level discovery: automatically creates items for discovered interfaces and disks
LSA / LSU	OSPF link-state advertisement / the update packet that carries LSAs
Native VLAN	The VLAN sent untagged on a trunk; set to unused 999 here
Network namespace	Linux isolated network stack; used here as virtual routers, switches and PCs
Passive interface	OSPF advertises the network but sends no hellos there
pcap	Packet-capture file format read by Wireshark and tshark
Router-on-a-stick	One router port with a subinterface per VLAN for inter-VLAN routing
Source of truth	The authoritative record of intended network state (NetBox)
Trunk	A link that carries multiple VLANs with tags
UFW	Uncomplicated Firewall: Ubuntu's simple firewall front end
WireGuard AllowedIPs	Which destination networks a peer may send to or receive from through the tunnel

> **Where to find everything:** README.md (overview) · docs/build-log.md (all steps and 23 problems) · testing/results/test-report.md (22 tests with evidence) · testing/troubleshooting/ (10 challenges) · screenshots/ (Packet Tracer, LibreNMS, Zabbix, NetBox, Grafana, Wireshark) · each tool folder has its own README with commands and results.